

Enterprise Systems and Services – Übungsprogramm

Jörn Kreutel– Berliner Hochschule für Technik – Sommersemester 2026

Stand: 4. März 2026

Inhaltsverzeichnis

SETUP (Vorbereitung von Entwicklungs- und Laufzeitumgebung)	3
BAS (Grundlagen: Frameworks, Reflection, Annotationen)	8
SER (HTTP und Ja*a EE Web Applikationen)	12
JRS (REST Web APIs mit JAX-RS)	18
WSV (Web Services Vertiefung)	24
MIP+JPA (Eclipse Microprofile und Ja*a Persistence API)	27
PAT (Architekturmuster für CDI und JPA)	37
MSD (Microservices Deployment)	41

Aufgaben

Lerneinheit	Übungskürzel	Übungsinhalt	Punkte	Abgabe	Pkt.Abg.	Anteil
BAS	BAS2	Reflection für Attribute und Methoden	4	1		
BAS	BAS3	Verwendung von Annotationen	3	1		
SER	SER3	Create Funktion für Service	5	1		
SER	SER4	Delete Funktion für Service	4	1		
JRS	JRS2	REST Service Implementierung	5	1		
JRS	JRS3	Polymorphie	3	1	24	0,282
WSV	WSV1	JAX-RS Client	14	3		
MIP+JPA	MIP+JPA1	Codegerüst der ProductCRUD Bean und Client-Zugriff	3	2		
MIP+JPA	MIP+JPA2	Partielle Implementierung der ProductCRUD Bean	4	2		
MIP+JPA	MIP+JPA3	Codegerüst der StockSystem Bean und Client-Zugriff	6	2		
MIP+JPA	MIP+JPA4	Partielle Implementierung der StockSystem Bean	4	2		
MIP+JPA	MIP+JPA5	Vervollständigung der Implementierung der ProductCRUD Bean	7	2		
MIP+JPA	MIP+JPA6	Vervollständigung der Implementierung der StockSystem Bean	7	2	31	0,365
PAT	PAT1	ShoppingSession mit PurchaseShoppingCartService	5	3		
PAT	PAT2	Erweiterter PurchaseShoppingCartService	6	3		
MSD	MSD1	Vorbereitung der Komponenten für Microservices-Deployment	5	3	30	0,353
			85		85	1

Grün hinterlegte Aufgaben sind Aufgaben, von deren Umsetzung die nachfolgenden Aufgaben der jeweiligen Lerneinheiten abhängen oder bei denen die volle Punktzahl nur aufgabenübergreifend erreicht werden kann (z.B. PAT1+PAT2).

Die Teilnahme an Abgabe 1 ist Voraussetzung für die Prüfungszulassung

Bei der Umsetzung der Aufgaben sollen die jeweils gefundenen Lösungswege durch Kommentare angemessen dokumentiert werden, andernfalls ist ein Punktabzug möglich. Dabei möge auch Gebrauch von schrittweisen Variablenzuweisungen als Alternative zu komplexen Einzeilern gemacht werden. Die Demos, die im Laufe der Veranstaltung gezeigt werden, können dafür als grober Anhaltspunkt genommen werden.

100% Bemessungsgrundlage für die Bewertung sind 85 Punkte.

SETUP

0. Voraussetzungen

- Verwenden Sie für alle Installationen und Workspaces keine Verzeichnisse, in deren Pfad Leerzeichen oder Sonderzeichen enthalten sind.
- Stellen Sie sicher, dass Sie ein Java JDK in Version 17 oder höher installiert haben. Die Kompilier- und Ausführbarkeit des in den Übungsaufgaben entwickelten Quellcodes wurde für das OpenJDK IBM Semeru in den Versionen 17 und 23 erfolgreich verifiziert.
 - Falls Sie keine entsprechende Version verfügbar haben oder falls beim Setup oder im Laufe der Veranstaltung Probleme auftreten, dann installieren Sie bitte das IBM Semeru OpenJDK in Version 17 innerhalb der IDEA Entwicklungsumgebung (siehe Punkt 3 unten).
- Stellen Sie sicher, dass die Versionsverwaltungssoftware *Git* auf Ihrem Entwicklungsrechner installiert ist. Installieren Sie andernfalls eine für Ihren Rechner geeignete Version: <https://git-scm.com/downloads>

1. IDEA installieren

- Laden Sie die JetBrains IDEA Entwicklungsumgebung in der *Ultimate Variante* herunter und installieren Sie sie: <https://www.jetbrains.com/idea/download>
- Starten Sie IDEA und nutzen Sie entweder die Evaluierungslizenz für 30 Tage oder registrieren Sie sich unter Ihrer Hochschuladresse bei JetBrains unter <https://www.jetbrains.com/student/> – für Adressen unter @bht-berlin.de sollten Sie freien Zugriff auf IDEA und andere JetBrains Produkte erhalten – falls hierbei Probleme auftreten, dann versuchen Sie es mit einer @beuth-hochschule.de Adresse.
- Zur Dokumentation von IDEA siehe <https://www.jetbrains.com/idea/documentation/>
- Für deutschsprachige Tastatur-Shortcuts siehe <https://victorvolle.wordpress.com/2012/05/16/intellij-german-keyboard-shortcuts-reference/> - sollten auch damit Probleme auftreten, können Sie Shortcuts individuell einrichten. Siehe dafür: <https://www.jetbrains.com/help/idea/configuring-keyboard-shortcuts.html>
- Im Zuge der Lehrveranstaltung benötigen Sie u.a. das ‘Tool Window’ *Maven*. Dieses können Sie über die Menüoption *View → Tool Windows...* öffnen. Siehe zu Tool Windows auch <https://www.jetbrains.com/idea/help/manipulating-the-tool-windows.html>
- Für die Bearbeitung des Übungsprogramms ist es empfehlenswert, die durch IDEA mittlerweile automatisch durchgeführte Importoptimierung – einstellbar durch die Option *Optimize Imports on the fly* – zu deaktivieren. Siehe dafür: <https://www.jetbrains.com/help/idea/settings-auto-import.html>

2. Implementierungsbeispiele aus Git laden

- IDEA zeigt Ihnen nach dem Starten eine Ansicht zum Öffnen, Erstellen und Laden von Projekten an.

- Wählen Sie die Option *Get from VCS* → *Git* und geben Sie als *Repository URL* die folgende URL ein: `https://github.com/dieschnittstelle/org.dieschnittstelle.ess.git`
 - Wenn Sie Ihren Quellcode selbst über GitHub verwalten möchten, dann richten Sie, falls noch nicht vorhanden, einen GitHub Account ein, erstellen Sie eine *Fork* des o.g. Projekts und geben Sie deren URL in IDEA ein. Siehe dafür `https://help.github.com/articles/fork-a-repo/`
- IDEA wird das ausgewählte Projekt laden und die Entwicklungsumgebung entsprechend den Projektanforderungen initialisieren. Dieser Vorgang kann mehrere Minuten dauern.
- Akzeptieren Sie insbesondere etwaige Aufforderungen bezüglich der Ausführung von Java und des Akzeptieren von eingehenden Netzwerkverbindungen.
- Aktivieren Sie keine weiteren Plugins, die IDEA Ihnen vorschlägt, insbesondere keine Plugins für Jakarta EE – die durch diese Plugins vorgenommenen Validierungen generieren mitunter irritierende Fehlermeldungen auf Error-Level, die auf den ersten Blick nicht von Fehlermeldungen des Java Compilers zu unterscheiden sind.

3. Java für Implementierungsbeispiele überprüfen/einrichten

- Öffnen Sie in IDEA die Projektkonfiguration unter *File* → *Project Structure* (siehe auch
- Überprüfen Sie unter *Project*, ob als *Project SDK* die bei Ihnen installierte JDK Version die Version 17 oder höher ist.
- Überprüfen Sie andernfalls, ob Ihnen die betreffende Version als Option angeboten wird und weisen Sie diese zu.
- Falls Sie noch kein entsprechendes JDK installiert haben, dann installieren Sie das *semeru-17* Open JDK direkt in IDEA, siehe dafür `https://www.jetbrains.com/help/idea/sdk.html#change-project-sdk`. Mit dieser Version wurde die Funktionsfähigkeit der Implementierungsbeispiele und des Übungsprogramms vorab erfolgreich verifiziert.
 - Sollten unter Windows beim Starten der Sever-Runtime Fehlermeldungen bezüglich der Nichtverfügbarkeit des Werts der `JAVA_HOME` Systemumgebungsvariablen auftreten, dann muss dieser Variable als Wert der Pfad zum Installationsverzeichnis des installierten JDK zugewiesen werden.

4. Implementierungsbeispiele kompilieren

- Die Implementierungsbeispiele enthalten alle Ressourcen, die Sie zur Umsetzung der Übungsaufgaben benötigen, sowie Konfigurationen zur Laufzeitausführung. Die für die Laufzeitausführung erforderlichen Laufzeitumgebung Apache TomEE wird als Bestandteil des Bauens der Anwendungen mit Apache Maven geladen und gestartet.
- Öffnen Sie *View* → *Tool Windows* → *Maven*. Ihnen wird eine Liste aller Teilprojekte der Implementierungsbeispiele angezeigt.

- Klappen Sie in der *Maven Projects* Ansicht im Projekt `build-all-modules` die *Lifecycle* Aktionen für dieses Projekt auf. Diese Aktionen dienen dazu, Projekte mit Maven zu bauen, d.h. den Quellcode zu kompilieren, eine JAR-Datei zu erstellen, diese ins lokale Maven Repository zu übertragen, etc. sowie ggf. gebaute Artefakte ‘aufzuräumen’.
- Führen Sie zunächst durch Doppelklick die Aktion `clean` und nach Abschluss die Aktion `install` aus. Damit sollten alle Projekte gebaut und die JAR Artefakte in Ihrem lokalen Maven Repository abgelegt werden.
 - Das lokale Maven Repository befindet sich normalerweise im Verzeichnis `.m2` im Home-Verzeichnis des auf Ihrem Entwicklungsrechner angemeldeten Users. Die JARs der Projekte werden in der dortigen Verzeichnisstruktur in den Verzeichnissen unter `repository/org/dieschnittstelle/ess` abgelegt.
- Für die einzelnen Aufgaben des Übungsprogramms wurden für IDEA sogenannte ‘Scopes’ deklariert. Diese können Sie nutzen, um die Projektansicht übersichtlich für die jeweils aufgabenrelevanten Teilprojekte zu filtern. Scopes können durch Öffnen des Aufklappmenüs in der Projektansicht oben links geöffnet werden. Beachten Sie, dass Ihnen bei Auswahl eines Scopes die Teilprojekte unter den Namen der zu den Teilprojekten gehörenden Maven-Artefakte angezeigt werden. Diese entsprechen in der Regel den Projektnamen nach dem `.esa` Segment, z.B. entspricht das Artefakt `ser-client` dem Projekt `org.dieschnittstelle.ess.ser.client`. Eine Ausnahme stellen die Projekte `.ser`, `.jrs`, `.jws` sowie `.mip` dar, deren Artefaktnamen zusätzlich den Jakarta EE Modultyp bezeichnen, d.h. `ser-webapp`, `jrs-webapp`, `jws-webapp` oder `mip-webapp`. Falls bei Auswahl eines Projekts die darin enthaltenen Klassen in einer entsprechend den Packages verschachtelten Ordnerstruktur angezeigt werden, können Sie die Ansicht durch Aktivierung der Option *Flatten Packages* optimieren, siehe hierfür: <https://stackoverflow.com/questions/22001171/how-to-expand-folded-package-chain-in-idea> bzw. <https://www.jetbrains.com/help/idea/project-tool-window.html>.
- Eine Dokumentation zur Integration von Maven in IDEA finden Sie hier: <https://www.jetbrains.com/help/idea/maven.html>
- Für weitere Hinweise zur Verwendung von Git in IDEA siehe: <https://www.jetbrains.com/help/idea/using-git-integration.html>

5. H2 Datenbank starten

- Um für die Aufgaben zu MIP+JPAff die von der Implementierung genutzte H2 Datenbank einzusehen, muss die Datenbank als Server gestartet werden und das durch den Server bereit gestellt Web Interface verwendet werden.
- Öffnen Sie in IDEA die Konfigurationsansicht unter *Run* → *Edit Configurations...* und wählen Sie die Konfiguration *H2 DB*.
- Weisen Sie als *Path to JAR* die Datei `h2-1.4.197.jar` im Unterverzeichnis `org.dieschnittstelle.ess.lib.utils/lib` Ihres Projektverzeichnisses zu, falls diese nicht bereits zugewiesen ist.
- Starten Sie den H2 Datenbankserver in IDEA durch Ausführung der *H2 DB* Konfiguration. Sie sollten ein User Interface im Browser erhalten, über das Sie sich anmelden können.

- Geben Sie als Credentials den Nutzernamen `sa` ein und verwenden Sie ein leeres Passwort. Geben Sie `jdbc:h2:tcp://localhost/~:/crm_erp_db` als Wert des Formularfelds 'JDBC URL' ein. **Achtung: Bei Übertragung der JDBC URL via Copy&Paste muss mindestens das Tilde-Symbol ('~') neu eingegeben oder ersetzt und müssen die Unterstriche ersetzt, d.h. ebenfalls neu eingegeben, werden!!!**
- **Das Starten der Datenbank auf dem genannten Wege ist erforderlich. Falls die Datenbank nicht gestartet wurde, schlägt der Start der Anwendungen fehl. Unter MacOS kann es evtl. erforderlich sein, einen unter dem Namen *Console* laufenden H2 Datenbankprozess manuell zu stoppen.**

6. Implementierungsbeispiele starten

- Um die grundsätzliche Funktionsfähigkeit der Implementierungsbeispiele auf Ihrem Rechner zu testen, führen Sie die folgenden Schritte aus:
 1. Starten Sie die JRS Run-Konfiguration und öffnen Sie nach Abschluss des Vorgangs, sobald der TomEE Server gestartet ist, die URL `http://localhost:8080`)
 2. Stoppen Sie den im vorangegangenen Schritt gestarteten Server und starten Sie, sofern sie noch nicht gestartet wurde, die H2 DB Konfiguration, dann die MIP-TOMEE Konfiguration und öffnen Sie nach Abschluss des Vorgangs, sobald der TomEE Server gestartet ist, die URL `http://localhost:8080`. Führen Sie dann `MIP+JPA: TotalUseCase` aus und steppen Sie die Ausführung in der Konsole durch, bis die Anwendung zum Abschluss kommt.

7. Allgemeine Hinweise

Falls bei der Ausführung von Run-Konfigurationen, insbesondere für Client-Anwendungen, unerwartete `ClassNotFoundException` Probleme auftreten, dann führen Sie im *Maven* Tool Window die Aktion *Reload All Maven Projects* aus. Sollte der Fehler auch danach noch vorhanden sein, führen Sie im selben Tool Window ein `clean` und danach ein `install` für das Projekt `build-all-modules` aus und wiederholen Sie anschließend die Aktion *Reload All Maven Projects*.

Die Maven `pom.xml` Dateien, in denen insbesondere die Abhängigkeiten deklariert werden, die für das Bauen und die Laufzeitausführung der einzelnen Projekte erforderlich sind, müssen und dürfen im Zuge der Bearbeitung des Übungsprogramms nicht geändert werden. Es gab in der Vergangenheit Fälle, in denen – angeregt durch die IDEA Entwicklungsumgebung – zusätzliche Abhängigkeiten zu `pom.xml` hinzugefügt wurden, die dann die Laufzeitausführung der Anwendungen beeinträchtigen. Besonders gravierend ist diese Problematik, falls versehentlich die `jakartae-web-api` als zusätzliche `compile` Dependency hinzugefügt wird. Dies verhindert ggf. den Start der betreffenden serverseitigen Anwendungen.

BAS

Projekte:

- `org.dieschnittstelle.ess.bas`

Server Runtime:

- (keine)

Ü BAS1 Methodenaufrufe via Reflection

Aufgabe

Erweitern Sie die Methode `buildStockItemFromElement` der Klasse `ReflectedStockItemBuilder` so, dass die in der XML Datei angegebenen Werte für `<price>` auf den erzeugten Instanzen von `IStockItem` gesetzt werden können.

Diese Aufgabe ist bereits umgesetzt. Machen Sie sich mit dem Programmcode vertraut.

Anforderungen

1. Verwenden Sie die Ausdrucksmittel der Reflection API, d.h. nehmen Sie an, dass Sie weder die beiden Klassen `Milk` und `Chocolate`, noch die Existenz eines `price` Attributs auf diesen Klassen kennen.

Bearbeitungshinweise

- Alle Attributnamen und Werte, die in der XML Datei verwendet werden sind in der lokalen `HashMap` namens `nodes` enthalten.
- Die Stelle, an der Sie die Ergänzungen vornehmen können, ist im Quellcode markiert.

Ü BAS2 Reflection für Attribute und Methoden**(4 Punkte)****Aufgabe**

Implementieren Sie die Methode `showAttributes()` der Klasse `ShowAnnotations`. Sie können hier und im folgenden die Ausdrucksmittel der Reflection API anhand eines einfachen Anwendungsfalls praktisch anwenden. Dieser Anwendungsfall findet sich in realen Anwendungen z.B. überall dort wieder, wo Java Objekte zwecks Übertragung über Netzwerkverbindungen in textueller Form, z.B. in JSON oder XML, repräsentiert werden müssen.

Anforderungen

1. Geben Sie für jedes Objekt, das der Methode übergeben wird, den Namen der Objektklasse sowie die durch die Objektklasse selbst deklarierten Attributnamen und deren Werte in der folgenden Form aus:

```
{<einfacher Klassenname> <attr1>:<Wert von attr1>, ...}, z.B.:  
{Milch menge:20, markenname:Mark Brandenburg}
```

2. Verwenden Sie die Ausdrucksmittel der Reflection API, d.h. nehmen Sie an, dass Sie die beiden Klassen `Milch` und `Schokolade` nicht kennen.

Ü BAS3 Verwendung von Annotationen**(3 Punkte)****Aufgabe**

Deklariieren Sie einen Annotationstyp `DisplayAs` mit Attribut `value`, der für Attribute von Klassen gesetzt und zur Laufzeit via Reflection ausgelesen werden kann.

Anforderungen

1. Ändern Sie die Implementierung der `showAttributes()` Methode aus Aufgabe 2 wie folgt: Wenn die `DisplayAs` Annotation für ein Attribut gesetzt ist, stellen Sie deren `value` Parameterwert anstelle des Attributsnamens dar, andernfalls verwenden Sie den Attributsnamen.
2. Setzen Sie die `DisplayAs` Annotation auf ausgewählten Attributen der Klassen `Milch` und `Schokolade`.

SER

Projekte:

- `org.dieschnittstelle.ess.ser`
- `org.dieschnittstelle.ess.lib.entities.crm` (SHARED)
- `org.dieschnittstelle.ess.lib.entities.erp` (SHARED)
- `org.dieschnittstelle.ess.ser.client` (Client)

Server Runtime:

- SER (TomEE)

Ü SER0 Überblick

Aufgabe

Hier finden Sie Erläuterungen zu den Implementierungsbeispielen.

Bearbeitungshinweise

- Die Implementierungsbeispiele bestehen aus einer Webanwendung (.ser bzw. ser-webapp) und einer Client-Anwendung (.ser.client) und illustrieren einige der in der Präsentation angesprochenen Funktionen von Jakarta EE Web Applikationen anhand zweier Szenarien:
 - Das TouchpointUIServlet steuert eine einfache graphische Nutzerschnittstelle zur Darstellung und Manipulation einer Menge existierender Verkaufsstellen (Touchpoints).- Hintergrund ist als E-Business-Szenario eine *Abverkaufskampagne für verderbliche Güter*.
 - * Der Aufbau des HTML Markups erfolgt mittels der JSP gui.jsp
 - * Der Zugriff auf das Servlet wird mittels des Filters TouchpointUIServletFilter auf Browser eingeschränkt.
 - Das TouchpointServiceServlet stellt eine programmatische Schnittstelle zur Verfügung.
 - * Als Client fungiert die Klasse ShowTouchpointService im Projekt .ser.client. *Die Ausführung dieser Klasse erfolgt schrittweise und lässt sich durch Betätigung der return-Taste in der Konsole steuern / 'steppen'. Beim erstmaligem Steppen ist evtl. eine zweifache Betätigung von return erforderlich.*
 - * Daten werden als serialisierte Java-Objekte ausgetauscht (in der Realität wäre das aufgrund der damit verbundenen Abhängigkeiten zwischen Client(s) und Webanwendung problematisch).
 - * Bisher können nur Daten ausgelesen werden – die Erweiterung ist Bestandteil des Übungsprogramms und kann mittels der JUnit Testklasse TestTouchpointService verifiziert werden.
- In beiden Szenarien wird für das Persistieren von Daten die Klasse TouchpointCRUDEXecutor verwendet. Diese wird durch den TouchpointServletContextListener verwaltet und über den ServletContext der Anwendung für alle Komponenten der Anwendung verfügbar gemacht.
- Eingehende Http Requests werden durch die Klasse HttpTrafficLoggingFilter als Logmeldung ausgegeben.

Ü SER1 Programmatischer Zugriff auf GUI Servlet

Aufgabe

Versuchen Sie, aus der Client Anwendung auf das Servlet zuzugreifen, das die graphische Nutzoberfläche steuert.

Bearbeitungshinweise

- Ersetzen Sie temporär in der Methode `readAllTouchpoints` der Client-Anwendung `ShowTouchpointService` die zugegriffene URI durch `http://localhost:8080/gui/touchpoints` und führen Sie die Anwendung von neuem aus.
- Was für einen Fehler bekommen Sie, und welche Komponente auf Seiten der Webanwendung ist dafür verantwortlich?
- Wie könnten Sie diese Komponente ‘überlisten’? – einen Hinweis dafür finden Sie in den Kommentaren der modifizierten Methode. Versuchen Sie, die Implementierung entsprechend zu ändern – sie werden aufgrund des nun nicht mehr verarbeitbaren Rückgabeformats aber einen client-seitigen Fehler bekommen.
- Machen Sie die Änderungen wieder rückgängig.

Ü SER2 Filter für Service

Aufgabe

Verhindern Sie, dass aus dem Browser auf das Servlet, das die Programmierschnittstelle zur Verfügung stellt, zugegriffen werden kann.

Bearbeitungshinweise

- Versuchen Sie von einem Browser aus auf die URI `http://localhost:8080/api/touchpoints` zuzugreifen. Ihnen sollte ein Download eines unbekannten Datenformats angeboten werden.
- Unterbinden Sie die Browser-Zugreifbarkeit auf die besagte URI durch Implementierung und Konfiguration einer geeigneten Jakarta EE Komponente. Sie können sich dafür an der bestehenden Umsetzung der Zugriffsverhinderung für programmatische Clients aus Ü1 orientieren.

Ü SER3 Create Funktion für Service**(5 Punkte)****Aufgabe**

Implementieren Sie die `createNewTouchpoint()` Methode der Klasse `ShowTouchpointService` durch Zugriff auf die Web Applikation und setzen Sie die `create`-Operation in der Klasse `TouchpointServiceServlet` um. Diese und die folgende Übung ermöglichen Ihnen das Kennenlernen client- und server-seitiger APIs zur Übermittlung und Bearbeitung von HTTP Requests in Java.

Anforderungen

1. Nutzen Sie für den HTTP Request, der durch den Client übermittelt wird die HTTP POST Methode.
2. Übermitteln Sie das server-seitig zu erstellende `AbstractTouchpoint` Objekt als serialisiertes Java-Objekt.
3. Nutzen Sie für die server-seitige Erstellung des Objekts die Methode `createTouchpoint()` auf `TouchpointCRUDEXecutor`.
4. Testen Sie Ihre Implementierung durch Ausführen von `ShowTouchpointService`.

Ü SER4 Delete Funktion für Service**(4 Punkte)****Aufgabe**

Implementieren Sie die `deleteTouchpoint()` Methode der Client-Klasse `ShowTouchpointService` durch Zugriff auf die Web Applikation und setzen Sie die delete-Operation in der Klasse `TouchpointServiceServlet` um.

Anforderungen

1. Nutzen Sie für den HTTP Request, der durch den Client übermittelt wird, die HTTP DELETE Methode.
2. Wählen Sie eine in Verbindung mit der genutzten HTTP Methode möglichst wenig redundante URI.
3. Nutzen Sie zur server-seitigen Ausführung des Löschens die Methode `deleteTouchpoint()` auf `TouchpointCRUDEXecutor`.
4. Zeigen Sie dem Client die erfolgreiche Ausführung des Löschens bzw. etwaige Fehler durch geeignete Status Codes im HTTP Response an.
5. Testen Sie Ihre Implementierung durch Ausführen von `ShowTouchpointService`.
6. Die gesamte Umsetzung von SER3 und SER4 können Sie nun durch Ausführen der Testklasse `TestTouchpointService` überprüfen.

Bearbeitungshinweise

- **Anforderung 1:** Sowohl der Apache HTTP Client, als auch die `HttpServlet` API unterstützen alle spezifizierten Methoden des HTTP Protokolls.
- **Anforderung 4:** Auf Client-Seite brauchen Sie dann keinen Rückgabewert auszulesen, sondern nur den Status aus dem Response zu berücksichtigen.

Sonstiges

Wenn die Löschaktion erfolgreich ausgeführt wurde, müssen Sie über das GUI eine neue Verkaufsstelle erzeugen – oder Sie implementieren Ü SER3.

JRS

Projekte:

- `org.dieschnittstelle.ess.jrs.api`
- `org.dieschnittstelle.ess.jrs`
- `org.dieschnittstelle.ess.lib.entities.crm` (SHARED)
- `org.dieschnittstelle.ess.lib.entities.erp` (SHARED)
- `org.dieschnittstelle.ess.jrs.client` (Client)

Server Runtime:

- JRS (TomEE)

Ü JRS1 Erweiterung der Beispielanwendung

Aufgabe

Machen Sie die `updateObject()` Methode aus `GenericCRUDEXecutor` über den in den Implementierungsbeispielen bereit gestellten REST Service für die Aktualisierung von `AbstractTouchpoint` Instanzen verfügbar.

Anforderungen

1. Erweitern Sie das Interface `ITouchpointCRUDService` und dessen Implementierung `TouchpointCRUDServiceImpl` um eine Methode, die den Zugriff auf die `updateObject()` Methode von `GenericCRUDEXecutor` ermöglicht.
2. Verwenden Sie geeignete JAX-RS Annotationen, um die neue Methode über den REST Service aufrufbar zu machen.
3. Rufen Sie die neue Methode in der Client-Implementierung `ShowTouchpointRESTService` auf dem dort verwendeten `serviceClient` Objekt auf.

Sonstiges

Die Umsetzung dieser Aufgabe gehört nicht zum Pflichtprogramm, dient aber dem Kennenlernen der Ausdrucksmittel von JAX-RS als Vorbereitung für die nachfolgenden Pflichtaufgaben.

Ü JRS2 REST Service Implementierung

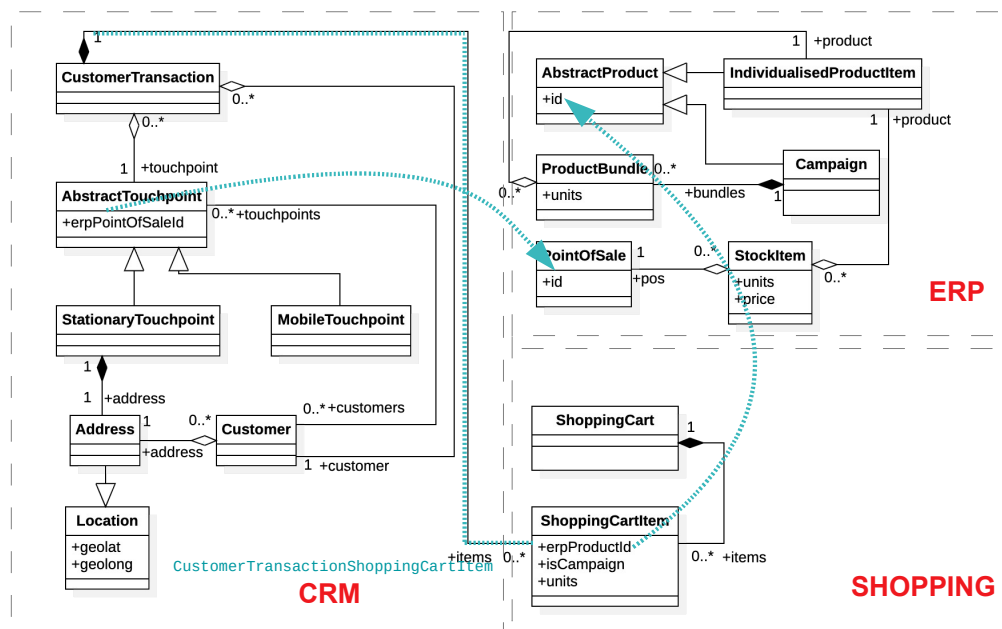
(5 Punkte)

Aufgabe

Implementieren Sie unter Verwendung der Ausdrucksmittel von JAX-RS einen Service, der die Erstellung neuer Produkte und das Auslesen aller Produkte ermöglicht und greifen Sie auf diesen Service aus einer Client Implementierung zu. Die Übung ermöglicht Ihnen, mit JAX-RS eine standardisierte Beschreibungsform für Web APIs anzuwenden, die eine empfehlenswerte Alternative zur händischen Umsetzung proprietärer Web APIs darstellt, wie sie in SER realisiert wurde.

Ausgangssituation

- Im Package `org.dieschnittstelle.ess.entities.erp` des Projekts `.lib.entities.erp` liegen mehrere Klassen, die ein Datenmodell für Produkte implementieren. Dieses finden Sie im nachfolgenden UML Diagramm rechts dargestellt.
 - Unterschieden werden einfache Produkte (`IndividualisedProductItem`) verschiedener Typen (`ProductType`), sowie Kampagnen (`Campaign`), die für einzelne Produkte in unterschiedlicher Stückzahl (`ProductBundle`) erstellt werden können. Als Abstraktion über `IndividualisedProductItem` und `Campaign` fungiert die Klasse `AbstractProduct`.
 - Mit diesem Datenmodell und seinen Erweiterungen werden sich auch die Übungsaufgaben in den kommenden Veranstaltungen beschäftigen.
- Im Package `org.dieschnittstelle.ess.jrs` im Projekt `.jrs.api` finden Sie das Interface `IProductCRUDService`, dessen Implementierung `ProductCRUDServiceImpl` das Projekt `.jrs` bzw. `jrs-webapp` enthält.
- Die Klasse `ProductCRUDRESTClient` im Client-Projekt `.jrs.client` enthält ein Grundgerüst für einen Resteasy Client bezüglich `IProductCRUDService` und wird von der JUnit Testklasse `TestProductRESTService` verwendet. Um auf den Service zugreifen zu können, müssen Sie noch analog zum Beispiel in `ShowTouchpointRESTService` das `serviceProxy` Instanzattribut instantiieren.
- Die Klasse `GenericCRUDExecutor` im Package `org.dieschnittstelle.ess.entities` des Projekts `.lib.entities.erp` generalisiert die Funktionalität von `TouchpointCRUDExecutor` aus der Veranstaltung SER und wird in der Beispielanwendung `.jrs` auf `AbstractTouchpoint` Instanzen angewendet. Die verwendete Instanz der Klasse wird in `TouchpointServletContextListener` erzeugt.
- Für Instanzen von `AbstractProduct` wird in `ProductServletContextListener` eine entsprechend Instanz von `GenericCRUDExecutor` erstellt und im `ServletContext` als Wert des Attributs `productCRUD` abgelegt.



Anforderungen

1. Implementieren Sie die Methoden in ProductCRUDServiceImpl unter Verwendung der Instanz von GenericCRUDEXecutor, die Sie als Wert des Attributs productCRUD aus dem ServletContext auslesen können.
2. Machen Sie die Implementierung durch Verwendung geeigneter JAX-RS Annotationen auf IProductCRUDService verfügbar. Nutzen Sie dafür die Wurzel-URI /products.
3. Instantiieren Sie im Client-Projekt das serviceProxy Attribut in ProductCRUDRESTClient.
4. Verifizieren Sie die Funktionalität und Zugreifbarkeit des Services durch Ausführung der Testklasse TestProductRESTService.

Bearbeitungshinweise

- **Anforderung 1:** Dafür können Sie ServletContext unter Verwendung der Annotation jakarta.ws.rs.core.Context im Konstruktor von ProductCRUDServiceImpl entgegen nehmen.
- **Anforderung 1:** Für die Rückgabe aller Produkte können Sie in ProductCRUDServiceImpl den Rückgabetypp von readAllObjects() auf GenericCRUDEXecutor auf den unspezifischen Typ List casten.
- **Anforderung 2:** Für den Zugriff auf Ihren Service wird diese Wurzel-URI als Segment an die URL `http://localhost:8080/api` hinzugefügt, die sich aus der URL des Tomcat Servers (`http://localhost:8080`) und dem gemeinsamen Wurzelsegment `/api` aller mittels JAX-RS bereit gestellter Services zusammensetzt. Letzteres wird in der Klasse WebAPIRoot deklariert.

Sonstiges

Greifen Sie auf die Auslese-Methode außerdem zu, indem Sie die hierfür zu verwendende URI im Browser eingeben. Ihnen sollte eine Liste von JSON Objekten angezeigt werden. Speichern Sie diese Liste in einer Textdatei ab.

Ü JRS3 Polymorphie

(3 Punkte)

Aufgabe

Verwenden Sie anstelle von `IndividualisedProductItem` die abstrakte Klasse `AbstractProduct` im Interface `IProductCRUDService` und in dessen Implementierung. Die Übung demonstriert, dass Web APIs bei Verwendung von Operationen, die abstrakte Klassen verwenden, Vorkehrungen treffen müssen, damit Instanzen konkreter Unterklassen überhaupt verarbeitet werden können.

Anforderungen

1. Ändern Sie die Methoden in `IProductCRUDService` und `ProductCRUDServiceImpl` so, dass Sie anstelle von `IndividualisedProductItem` die Klasse `AbstractProduct` verwenden.
2. Stellen Sie dann die Funktionsfähigkeit des Services wieder her.
3. Aktivieren Sie den Test `f_createCampaign()` in `TestProductRESTService` und die Methode `createCampaign()` in `ProductCRUDRESTClient`, und verifizieren Sie die Funktionsfähigkeit des Services durch Ausführen der Testklasse `TestProductRESTService`.

Bearbeitungshinweise

- **Anforderung 1** Führen Sie nun `TestProductRESTService` von neuem aus. Was für einen Fehler bekommen Sie?
- **Anforderung 2** Entfernen Sie dafür in der Klasse `AbstractProduct` die Auskommentierung der `@JsonTypeInfo` Annotation sowie die der beiden Annotationen `@JsonbTypeDeserializer` und `@JsonbTypeSerializer` sowie in `IndividualisedProductItem` und `Campaign` die Auskommentierung von `@JsonbTypeSerializer`. Nach Neustart der Webanwendung sollte bei erneutem Zugriff mittels `TestProductRESTService` der Fehler nicht mehr auftreten.

Sonstiges

Greifen Sie nun mit dem Browser auf die URL für das Auslesen aller Produkte zu und vergleichen Sie die Ausgabe mit der in Ü JRS2 gespeicherten. Was hat sich strukturell an der Repräsentation der Daten geändert?

WSV

Projekte:

- `org.dieschnittstelle.ess.wsv.client` (Client-Codegerüst)
- `org.dieschnittstelle.ess.lib.entities.erp` (SHARED)
- `org.dieschnittstelle.ess.lib.entities.crm` (SHARED)

Server Runtime:

- JRS (TomEE)

Ü WSV1 JAX-RS Client

(14 Punkte)

Aufgabe

Implementieren Sie einen `InvocationHandler` der Ihnen für ein JAX-RS annotiertes Interface den Zugriff auf einen REST Service ermöglicht, welcher das Interface bereit stellt. Diese Übung verleiht Ihnen Einblick in die Mechanismen, auf denen die Implementierungen generischer Client-Frameworks für JAX-RS wie das in JRS verwendete Resteasy Framework basieren.

Anforderungen

1. Der Funktionsumfang des `InvocationHandler` soll es Ihnen ermöglichen, die 5 CRUD Operationen aufzurufen, die im Interface `ITouchpointCRUDServiceClient` des `ess.wsv` Projekts mittels JAX-RS Annotationen deklariert werden. **(10 Punkte)**
 - Ihre Implementierung des `InvocationHandler` darf keine Abhängigkeiten zu Klassen der Implementierungsbeispiele für `ess.wsv` mit Ausnahme der Klassen für die Umwandlung von JSON Objekten in Java-Objekte enthalten, d.h. Abhängigkeiten zu `ITouchpointCRUDServiceClient`, `StationaryTouchpoint` etc. sind dort nicht erlaubt.
 - Wenn Sie die (freiwillige) Aufgabe JRS1 noch nicht umgesetzt haben, dann fügen Sie die `updateTouchpoints()` Methode zum Interface `ITouchpointCRUDService` im Projekt `ess.jrs.api` und zu dessen Implementierung in `ess.jrs` hinzu, um die server-seitige Implementierung zu vervollständigen. Das in WSV client-seitig verwendete Interface gleichen Namens enthält die Methode bereits.
 - Verifizieren Sie die Funktionsfähigkeit Ihrer Implementierung durch Zugriff auf den JAX-RS Web Service für Touchpoints aus dem `.jrs` bzw. `jrs-webapp` Projekt. Verwenden Sie dafür die Klasse `AccessRESTServiceWithInterpreter`. **Die Ausführung dieser Klasse erfolgt schrittweise und lässt sich durch Betätigung der `return`-Taste in der Konsole steuern / 'steppen'. Beim erstmaligem Steppen ist evtl. eine zweifache Betätigung von `return` erforderlich.**
2. Ersetzen Sie in allen Methodensignaturen der lokalen Deklaration von `ITouchpointCRUDServiceClient` im Projekt `.wsv.client` den Typ `StationaryTouchpoint` durch `AbstractTouchpoint`. Stellen Sie dann die volle Funktionsfähigkeit von `AccessRESTServiceWithInterpreter` wieder her. **(4 Punkte)**

Bearbeitungshinweise

- Ein 'Gerüst' für die Implementierung finden Sie im Projekt `.wsv.client`. Dort wird das Service Interface `ITouchpointCRUDServiceClient` lokal deklariert. Um die `update` Methode ausführen zu können, müssen Sie diese noch entsprechend der – nicht selbständig bepunkteten – Aufgabe JRS1 server-seitig umsetzen.
- Darin finden Sie im Package `.ess.wsv.interpreter` u.a. ein Codegerüst für einen `InvocationHandler` in `JAXRSClientInterpreter` sowie die Klassen `JSONObjectSerialiser` und `JSONObjectMapper`, mit der Sie die Abbildung von Instanzen anwendungsspezifischer Klassen auf JSON Objekte vornehmen können und umgekehrt.

- Für die Ausführung von HTTP Requests können Sie in der Implementierung von `JAXRSCientInterpreter` die Apache HTTP Client API nutzen, die Sie auch in SER verwendet haben. Die Imports hierfür sind bereits im `pom.xml` vorhanden.
- **Anforderung 2** Dafür müssen Sie eine Ergänzung an der mit *TODO* markierten Stelle in `JSONObjectMapper` vornehmen sowie ggf. die Annotation `JsonTypeInfo` auf `AbstractTouchpoint` einkommen-tieren.

MIP+JPA

Projekte:

- **org.dieschnittstelle.ess.mip.components.erp.api**
- **org.dieschnittstelle.ess.mip.components.erp.impl**
- org.dieschnittstelle.ess.mip.components.crm.api
- org.dieschnittstelle.ess.mip.components.crm.impl
- org.dieschnittstelle.ess.mip.components.shopping.api
- org.dieschnittstelle.ess.mip.components.shopping.impl
- org.dieschnittstelle.ess.mip.webapp
- org.dieschnittstelle.ess.lib.entities.crm
- **org.dieschnittstelle.ess.lib.entities.erp**
- org.dieschnittstelle.ess.lib.utils
- org.dieschnittstelle.ess.mip.app.tomee (Deployment)
- **org.dieschnittstelle.ess.mip.client** (Client)

Server Runtime:

- MIP-TOMEE (TomEE)

Ü MIP+JPA0 Implementierungsbeispiele

Aufgabe

Nachfolgend finden Sie einen Überblick über den Aufbau der Implementierungsbeispiele, die als Grundlage der Übungsaufgaben im Themengebiet MIP+JPA dienen.

Ausgangssituation

- Die Beispielanwendung umfasst Komponenten für die drei Domains `erp`, `crm` und `shopping`, die jeweils in ein `.api` und ein `.impl` Projekt aufgegliedert sind. Diese Projektorganisation ermöglicht es uns, die server-seitigen Anwendungskomponenten wahlweise innerhalb einer einzigen Server-Instanz in Betrieb zu nehmen (MIP-TOMEE oder alternativ MIP-WILDFLY) oder sie als drei separat voneinander laufende Microservices zu betreiben (MIP-TOMEE-MS-ERP|CRM|SHOPPING).
- Der Remote Zugriff auf die Komponenten erfolgt aus mehreren einfachen Java SE Anwendungen im Projekt `.mip.client` und verwendet dafür die durch den Server bereit gestellt Web API, auf die mittels der Resteasy Bibliothek zugegriffen wird (siehe dafür JRS).
- Die Komponenten, die sich pro Domain in den mit `.crud` und `.crud.impl` bezeichneten Packages befinden, deklarieren lesende und schreibende Zugriffe auf die von der Anwendung genutzten Datenbestände bzw. setzen diese Zugriffe durch Aufruf einer relationalen H2 Datenbank um. Dafür verwenden Sie die Ausdrucksmittel der *Ja*a Persistence API* (JPA).
- Umgesetzt sind u.a. die folgenden client-seitigen Use Cases, die in der Klasse `TotalUsecase` gebündelt sind:
 - Erstellung von Touchpoints (`TotalUsecase.createTouchpoints()`)
 - Erstellung von Kunden (`TotalUsecase.createCustomers()`)
 - Einrichtung von Kampagnen (`TotalUsecase.prepareCampaigns()`)
 - Befüllung eines Warenkorbs und (partielle) Durchführung eines Kaufs (`TotalUsecase.doShopping()`)
 - Nachverfolgen der Einkäufe eines Kunden (`TotalUsecase.doShopping()`, `TotalUsecase.showTransactions()`)
- Für `TotalUsecase.doShopping()` wird die client-seitige Implementierung einer `ShoppingSession` verwendet, deren Funktionsumfang partiell dem Beispiel im Lehrmaterial entspricht.
- Gegenstand der Übungsaufgaben ist insbesondere die Implementierung der folgenden Funktionen der `erp` Domain, die mit den jeweils genannten Testklassen verifiziert werden können:
 - Erstellung, Manipulation und Abfragen von Produkten (`TestProductCRUD`)
 - Erstellung, Manipulation und Abfragen von Lagerbeständen (`TestStockSystem`)
- Die nachfolgenden Übungsaufgaben dienen der schrittweisen Entwicklung der durch die Testklassen überprüften Funktionen. Sie orientieren sich an den für den SU der Veranstaltung vorgesehenen Demos. Die im weiteren Verlauf umzusetzenden Aufgaben PAT1 und PAT2 werden die Einbindung dieser Funktionen, insbesondere den Zugriff auf Lagerbestände beim Abschluss eines Kaufvorgangs, in `TotalUsecase` beinhalten.

Ü MIP+JPA1 Codegerüst der ProductCRUD Bean und Client-Zugriff (4 Punkte)

Aufgabe

Erstellen Sie ein Gerüst für die Implementierung des Interfaces ProductCRUD und greifen Sie von Client-Seite aus auf dieses zu. Der Ausbau des Codegerüsts wird in den Aufgaben Ü MIP+JPA2 sowie Ü MIP+JPA5 erfolgen.

Anforderungen

1. Versehen Sie ProductCRUD in `.mip.components.erp.api` mit JAX-RS Annotationen zur Bereitstellung im Rahmen der server-seitigen Web API.
2. Implementieren Sie ProductCRUD als transaktionale CDI Bean mit Application Scope in `.mip.components.erp.impl`. Lassen Sie `createProduct()` und `readAllProducts()` nicht-null Werte zurückgeben.
3. Setzen Sie auf AbstractProduct, falls noch nicht vorhanden, die `JsonbTypeSerializer` und `JsonbTypeDeserializer` Annotation und auf IndividualisedProductItem die `JsonbTypeSerializer` Annotation, um die JSON Serialisierung der Instanzen dieser Klasse mit Typinformation auszustatten bzw. den konkreten Typ aus einer JSON Serialisierung zu ermitteln. Verwenden Sie dafür als Vorlage die Klassen `AbstractTouchpoint` und `StationaryTouchpoint`.
4. Nehmen Sie die Änderungen durch Starten der Run Konfiguration MIP-TOMEE in Betrieb; starten Sie zuvor, falls erforderlich, die H2 Datenbank.
5. Verwenden Sie ProductCRUD in ProductCRUDClient, um auf die Web API zuzugreifen. Vervollständigen Sie dafür die Instantiierung des Attributs `serviceProxy` und aktivieren Sie die auskommentierten Codestellen, in denen das Objekt `serviceProxy` verwendet wird. Kommentieren Sie insbesondere auch die manuelle ID Zuweisung in `createProduct()` aus.
6. Führen Sie die Client-Anwendung ShowStockSystem aus, um den Zugriff zu testen. Die Ausführung dieser Klasse erfolgt schrittweise und lässt sich durch Betätigung der *return*-Taste in der Konsole steuern / 'steppen'. Beim erstmaligem Steppen ist evtl. eine zweifache Betätigung von *return* erforderlich.

Bearbeitungshinweise

- Beachten Sie, dass die Annotationsklassen für `@PathParam` aus dem `jakarta.ws.rs` Package importiert werden. Andernfalls treten Laufzeitfehler auf.
- **Anforderung 5** Dafür müssen Sie im Konstruktor der Klasse das Instanzattribut `serviceProxy` durch Verwendung der `getProxy()` Methode von `ServiceProxyFactory` instantiieren.
- Der hier verwendete Begriff der 'Domain' orientiert sich grob an der Idee des *Domain-driven Designs* (siehe z.B. Vaughn Vernon. *Implementing domain-driven design*. Addison-Wesley, 2013) Bei den drei hier angenommenen Domains `crm`, `erp` und `shopping` handelt es sich um Sub-Domains des Anwendungsbereichs – der Domain – einer typischen E-Commerce-Anwendung. Mit Blick auf die technische Umsetzung einer Anwendung für diese Domain entsprechen die drei Bereiche jeweils einem *Bounded Context* mit eigenen Entitäten und geschäftslogischen

Operationen. Da der ‘Kontext’-Begriff in der von uns verwendeten CDI Technologie jedoch in einem anderen Sinne als dem des Bounded Context gebraucht wird, nutzen Übungsprogramm und Präsentationsfolien den Begriff der Domain, um einer möglichen Begriffskonfusion vorzubeugen.

Ü MIP+JPA2 Partielle Implementierung der ProductCRUD Bean

(4 Punkte)

Aufgabe

Greifen Sie aus der ProductCRUD Implementierung auf die H2 Datenbank zu.

Anforderungen

1. Deklarieren Sie `IndividualisedProductItem` als JPA Entity, inklusive aller dafür erforderlichen Annotationen.
2. Fügen Sie `IndividualisedProductItem` und `AbstractProduct` der `erp_PU` Persistence Unit hinzu.
3. Verwenden Sie in Ihrer Implementierung von `ProductCRUD` den JPA `EntityManager`. Deklarieren Sie dafür ein geeignetes Instanzattribut und fordern Sie eine Wertzuweisung mittels Dependency Injection durch Setzen der dafür vorgesehenen CDI Annotation an. Verwenden Sie den korrekten CDI Qualifier, um den Entity Manager für die Persistence Unit `erp_PU` zu erhalten.
4. Starten Sie den MIP-TOMEE Server neu.
5. Führen Sie die Client-Anwendung `ShowStockSystem` aus, um die Implementierung zu testen.
6. Falls Schritt 5 ohne Exceptions ausgeführt werden kann, dann öffnen Sie die Konsole der H2 Datenbank (via `http://localhost:8082`) und schauen Sie sich die Inhalte der `ABSTRACTPRODUCT` Tabelle an.

Bearbeitungshinweise

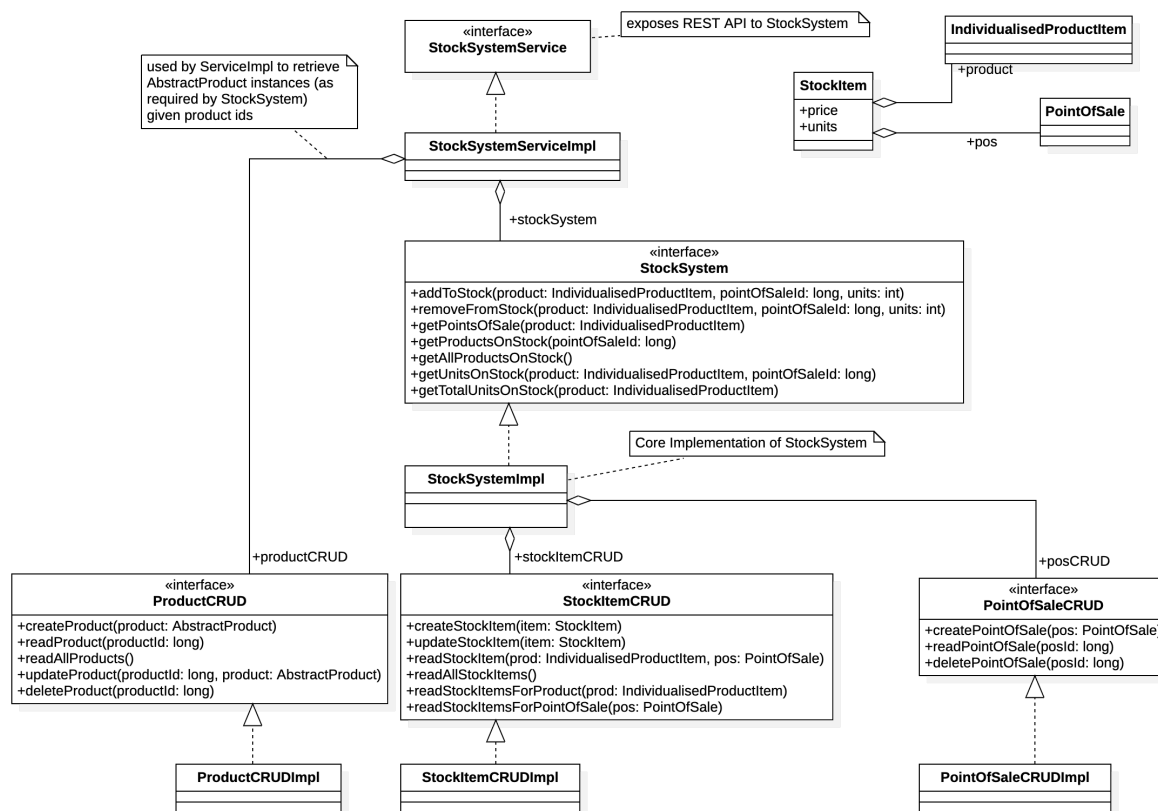
- **Anforderung 2** Dafür müssen die beiden Klassen in der `persistence.xml` Konfigurationsdatei des `.mip.components.erp.impl` Projekts als zu dieser Persistence Unit gehörig deklariert werden.

Ü MIP+JPA3 Codegerüst der StockSystem Bean und Client-Zugriff

(6 Punkte)

Aufgabe

Erstellen Sie ein Gerüst für die Implementierung des Interfaces `StockSystem` als CDI Bean. Verwenden Sie darin das CRUD Interface für `StockItem` und greifen Sie von Client-Seite aus auf die `StockSystem` Bean zu, indem Sie das Interface `StockSystemService` mit JAX-RS Annotationen ausstatten und implementieren. Der Ausbau des Codegerüsts wird in den Aufgaben Ü MIP+JPA4 sowie Ü MIP+JPA6 erfolgen. Das folgende UML Diagramm stellt Funktionalität und Abhängigkeiten der für die Umsetzung zu verwendenden Komponenten dar.



Anforderungen

1. Implementieren Sie `StockItemCRUD` als CDI Bean mit Application Scope im Projekt `.mip.components.erp.impl` behalten Sie dabei zunächst die von der Entwicklungsumgebung generierten Default-Implementierungen der vorgesehenen Methoden bei.
2. Implementieren Sie `StockSystem` als CDI Bean mit Application Scope im Projekt `.mip.components.erp.impl`; behalten Sie dabei zunächst die Default-Implementierungen der Methoden bei.
3. Deklarieren Sie in Ihrer `StockSystem` Implementierung Instanzattribute für `StockItemCRUD` und `PointOfSaleCRUD` und fordern Sie deren Setzung mittels Dependency Injection an.
4. Versetzen Sie `StockSystemService` mit JAX-RS Annotationen zur Bereitstellung im Rahmen der server-seitigen Web API.

5. Erstellen Sie eine Implementierung von `StockSystemService` als CDI Bean im Projekt `.mip.components.erp.in` und greifen Sie darin mittels Dependency Injection auf die `StockSystem` Bean zu. Nutzen Sie für das Auslesen von Produkten für eine gegebene `id` die `ProductCRUD` Bean. Evtl. ist dafür noch die Implementierung der `readProduct()` Methode in `ProductCRUDImpl` erforderlich.
6. Starten Sie, falls erforderlich, die H2 Datenbank und dann die Run Konfiguration für MIP-TOMEE.
7. Verwenden Sie `StockSystemService` in `StockSystemClient`. Vervollständigen Sie dafür die Instantiierung des Attributs `serviceProxy` und aktivieren Sie alle Codestellen, in denen auf `serviceProxy` zugegriffen wird.
8. Führen Sie `ShowStockSystem` aus und suchen Sie im JBoss Log die Logmeldungen, anhand derer Sie den Zugriff auf die `StockSystem` Bean nachvollziehen können.

Ü MIP+JPA4 Partielle Implementierung der StockSystem Bean

(4 Punkte)

Aufgabe

Ermöglichen Sie das Erstellen neuer und die Aktualisierung bestehender Lagerbestände durch partielle Implementierung der beiden in Ü MIP+JPA3 erstellten Beans.

Anforderungen

1. Deklarieren Sie StockItem als Entity, inklusive aller dafür erforderlichen Annotationen.
2. Implementieren Sie in Ihrer StockSystem Bean die addToStock() Methode unter Verwendung von PointOfSaleCRUD und StockItemCRUD. Weitere Hinweise zur Umsetzung dieser und anderer Methoden finden Sie in den Kommentaren auf StockSystem.
3. Lassen Sie Ihre StockItemCRUD Bean einen EntityManager für die Persistence Unit erp_PU verwenden.
4. Implementieren Sie in der StockItemCRUD Bean die readStockItem() Methode. Verwenden Sie zur Identifikation des auszulesenden StockItem einen JPQL Query, in dem Sie die id-Werte des Produkts und des Point of Sale verwenden.
5. Implementieren Sie die createStockItem() Methode der StockItemCRUD Bean.
6. Starten Sie die MIP-TOMEE Run Konfiguration neu, ggf. unter vorherigem Starten der H2 Datenbank.
7. Führen Sie ShowStockSystem aus. Falls Fehler auftreten, dann überprüfen Sie die Implementierung von createStockItem() u.a. unter Berücksichtigung der Hinweise im Lehrmaterial zu Entities, die andere Entities verwenden.

Bearbeitungshinweise

- **Anforderung 2** Zur Ermittlung eines PointOfSale für eine gegebene pointOfSaleId kann Ihre StockSystem Implementierung das Interface PointOfSaleCRUD verwenden und das daraus ausgelesene PointOfSale Objekt an StockItemCRUD übergeben.

Ü MIP+JPA5 Vervollständigung der Implementierung der ProductCRUD Bean (7 Punkte)

Aufgabe

Implementieren Sie die bisher noch nicht ausgeführten Methoden der ProductCRUD Bean und machen Sie diese für IndividualisedProductItem und Campaign funktionsfähig.

Anforderungen

1. Deklarieren Sie Campaign als JPA Entity, inklusive aller dafür erforderlichen Annotationen.
2. Deklarieren Sie die mit Campaign assoziierten Klassen als JPA Entities, inklusive aller dafür erforderlichen Annotationen.
3. Fügen Sie alle deklarierten Entities der `erp_PU` Persistence Unit hinzu.
4. Implementieren Sie die noch nicht umgesetzten Methoden aus ProductCRUD.
5. Für die Umsetzung der Methode `getCampaignsForProduct()` soll dabei ein JPQL Query mit JOIN Operator verwendet werden
6. Führen Sie die Testklasse `TestProductCRUD` aus, um die Implementierung zu testen. Nehmen Sie ggf. erforderliche Nachbesserungen vor. Wenn die Testklasse erfolgreich ausgeführt werden kann, ist die Umsetzung der Aufgabe abgeschlossen.

Bearbeitungshinweise

- **Anforderung 6** Nachbesserungen sind evtl. u.a. bezüglich der mit JPA Annotationen auszudrückenden Modellierung der Assoziation von Campaign und ProductBundle als Komposition erforderlich.
- **Anforderung 5** Ein Beispiel für einen solchen Query finden Sie in der Methode `readAllTransactionsForProduct()` der Klasse `CustomerTransactionCRUDImpl`.
- Nach jedem Neustart der server-seitigen Anwendung werden beim ersten Zugriff auf eine der Tabellen der voneinander getrennten Schemata für CRM, ERP und SHOPPING alle zu dem betreffenden Schema gehörenden Tabellen jeweils zurückgesetzt und neu erstellt. Sollten Sie eine Fehlermeldung *Cannot drop...* beim Löschen einer Tabelle erhalten, die sich auf die Verletzung abhängiger *Constraints* bezieht, dann setzen Sie die Datenbank manuell zurück. Öffnen Sie dafür die H2 Konsole unter `http://localhost:8082/`, greifen Sie mit den im Setup auf S. 6 genannten Angaben auf die Datenbank zu und führen Sie den Befehl `drop all objects` aus. Starten Sie danach den MIP-TOMEE Server neu.

Ü MIP+JPA6 Vervollständigung der Implementierung der StockSystem Bean (7 Punkte)

Aufgabe

Implementieren Sie die bisher noch nicht ausgeführten Methoden der StockSystem Bean inklusive der dafür erforderlichen Methoden der StockItemCRUD Bean.

Anforderungen

1. Implementieren Sie alle im StockSystem Interface vorgesehenen Methoden in Ihrer Implementierung des Interfaces.
2. Der direkte Zugriff auf die Datenhaltungsschicht ist in der StockSystem Bean nicht zulässig. Setzen Sie für die erforderlichen Datenzugriffe die noch nicht ausgeführten Methoden des StockItemCRUD Interfaces in der dazugehörigen Bean um.
3. Starten Sie nach erfolgter Umsetzung den MIP-TOMEE Server neu.
4. Führen Sie die Testklasse TestStockSystem aus, um die Implementierung zu testen. Nehmen Sie ggf. erforderliche Nachbesserungen vor. Wenn die Testklasse erfolgreich ausgeführt werden kann, ist die Umsetzung der Aufgabe abgeschlossen.

Bearbeitungshinweise

- *Anforderung 1/Anforderung 2* Die Interfaces StockSystem und StockItemCRUD enthalten ausführliche Hinweise zur Implementierung der beiden Beans entsprechend den Anforderungen.

PAT

Projekte:

- `org.dieschnittstelle.ess.mip.components.erp.api`
- `org.dieschnittstelle.ess.mip.components.erp.impl`
- `org.dieschnittstelle.ess.mip.components.crm.api`
- `org.dieschnittstelle.ess.mip.components.crm.impl`
- **`org.dieschnittstelle.ess.mip.components.shopping.api`**
- **`org.dieschnittstelle.ess.mip.components.shopping.impl`**
- `org.dieschnittstelle.ess.mip.webapp`
- `org.dieschnittstelle.ess.lib.entities.crm`
- `org.dieschnittstelle.ess.lib.entities.erp`
- `org.dieschnittstelle.ess.lib.utils`
- `org.dieschnittstelle.ess.mip.app.tomee (Deployment)`
- **`org.dieschnittstelle.ess.mip.client`** (Client)

Server Runtime:

- MIP-TOMEE (TomEE)

[illegible]

1. Verwenden Sie für die Umsetzung die Interfaces und Klassen / Codegerüste aus dem Package `org.dieschnittstelle.ess.mip.client.shopping` des Client-Projekts und die serverseitigen Komponenten aus den Projekten `org.dieschnittstelle.ess.mip.components.shopping.api|impl`.
2. Versetzen Sie `PurchaseService` mit JAX-RS Annotationen zur Bereitstellung im Rahmen der server-seitigen Web API.
3. Übertragen Sie die Funktionalität, die die Klasse `ShoppingSession` aus der Client-Anwendung implementiert, in eine Bean mit Request Scope, die das Interface `PurchaseService` implementiert, und entfernen Sie den Konstruktor sowie die Methoden `setCustomer()`, `setTouchpoint()`

und `addProduct()`.

4. Nutzen Sie in Ihrer neuen Bean für den Zugriff auf die von ihr benötigten Beans die Dependency Injection Mechanismen, die Ihnen CDI bereitstellt.
5. Rufen Sie in Ihrer Implementierung der Methode `purchaseCartAtTouchpointForCustomer()` die `purchase()` Methode auf, die Sie aus `ShoppingSession` übertragen haben.
6. Greifen Sie aus der Client-seitigen Klasse `PurchaseServiceClient` auf die neue Bean zu.
7. Verwenden Sie in `TotalUsecase.doShopping()` die Klasse `PurchaseServiceClient` anstelle von `ShoppingSession`.

Bearbeitungshinweise

- **Anforderung 7:** die betreffende Stelle ist im Quellcode mit Hinweis auf PAT1 markiert – sie müssen lediglich das Attribut `usePurchaseServiceClient` auf `true` setzen. Diese Änderung können Sie für das weitere Übungsprogramm beibehalten.

Ü PAT2 Vervollständigung von PurchaseService**(6 Punkte)****Aufgabe**

Vervollständigen Sie die in PAT1 begonnene Implementierung von PurchaseService, indem Sie die im Zuge eines Einkaufs erworbenen Produkte aus dem Warenlager entfernen.

Anforderungen

1. Implementieren Sie die in der purchase() Methode aufgerufene Methode checkAndRemoveProductsFromStock() in Ihrer Implementierung von PurchaseService entsprechend den im Code vorgegebenen Instruktionen.
2. Testen Sie Ihre Implementierung anhand der Testklasse TestShoppingSession.

Bearbeitungshinweise

- **Anforderung 1** Dafür muss Ihre PurchaseService Bean zusätzlich zu den in PAT1 verwendeten Beans noch Abhängigkeiten zur ProductCRUD Bean sowie zur StockSystem Bean deklarieren.

MSD

Projekte:

- `org.dieschnittstelle.ess.mip.components.erp.api`
- `org.dieschnittstelle.ess.mip.components.erp.impl`
- `org.dieschnittstelle.ess.mip.components.crm.api`
- `org.dieschnittstelle.ess.mip.components.crm.impl`
- `org.dieschnittstelle.ess.mip.components.shopping.api`
- `org.dieschnittstelle.ess.mip.components.shopping.impl`
- `org.dieschnittstelle.ess.mip.webapp`
- `org.dieschnittstelle.ess.lib.entities.crm`
- `org.dieschnittstelle.ess.lib.entities.erp`
- `org.dieschnittstelle.ess.lib.utils`
- `org.dieschnittstelle.ess.mip.ms.erp.app.tomee` (Deployment)
- `org.dieschnittstelle.ess.mip.ms.crm.app.tomee` (Deployment)
- `org.dieschnittstelle.ess.mip.ms.shopping.app.tomee` (Deployment)
- `org.dieschnittstelle.ess.mip.client` (Client)

Server Runtime:

- MIP-TOMEE (TomEE)

Ü MSD1 Vorbereitung und Inbetriebnahme der Microservices

(5 Punkte)

Aufgabe

Bereiten Sie die bisher genutzten Implementierungskomponenten für die Inbetriebnahme in drei separaten Microservices vor. Nach Umsetzung der Änderungen müssen beide Deployment-Varianten lauffähig sein.

Anforderungen

1. Fordern Sie für alle Bean Interfaces aus den .api Projekten für `erp`, `crm` und `shopping`, die in anderen als ihrer eigenen Domain genutzt werden, die Generierung von REST Clients an.
2. Geben Sie für alle Implementierungen der in **Anforderung 1** modifizierten Bean Interfaces an, dass diese für den Fall der Verfügbarkeit mehrerer alternativer Implementierungen eines Bean Interfaces als präferierte Alternative ausgewählt werden sollen.
3. Starten Sie nach Umsetzung der Änderungen das all-in-one-Deployment mittels MIP-TOMEE oder MIP-WILDFLY und überprüfen Sie die Funktionsfähigkeit der bisher, d.h. vor Umsetzung der Änderungen, laufenden Anwendung durch Ausführung von `TotalUsecase`. Sollten Probleme auftreten, dann nehmen Sie ggf. notwendige Korrekturen vor.
4. Geben Sie in den `mip.ms.*` Projekten für die drei Domains `erp`, `crm` und `shopping` die Wurzel-URLs der Web APIs an, über die die jeweils erforderlichen Beans der anderen Domains zugegriffen werden können.
5. Starten Sie dann die drei Microservices für die Domains `erp`, `crm` und `shopping` und testen Sie die Funktionsfähigkeit des Microservice-Deployments wahlweise durch Ausführung von `TotalUsecaseMS` oder, falls Sie PAT2 schon umgesetzt haben, anhand der Testklasse `TestShoppingSessionMS`.

Bearbeitungshinweise

- Beachten Sie, dass die Server für die beiden Deployment-Varianten zwar gemeinsam gestartet werden können, dass aber bei wechselseitigem Zugriff auf das all-in-one vs. das Microservices-Deployment Fehler zu erwarten sind. Bevor Sie eine der beiden Varianten testen, sollten daher der bzw. die dafür erforderlichen Server jeweils neu gestartet werden. Ungeachtet dessen können für das Testen der Microservices-Variante die drei dafür erforderlichen Server unabhängig voneinander einzeln gestartet bzw. aktualisiert werden.